Synchronous ADC Interfacing on the AMD Kria KR260

Daniel Mejia

Abstract

The interface between the analog-to-digital converter (ADC) and the AMD Kria KR260 currently represent one of the highest risk components of our proposed direction finding system. This paper is intended to investigate various communication approaches considered during initial designs, the engineering tradeoffs that led to the rejection of a certain approach, and the technical challenges the current design still faces. This paper will have a particular focus on the requirements for reliable ADC data acquisition and the methods being considered to resolve these challenges.

I. Introduction

The aim of the Saab sponsored senior design project aims to develop a radio frequency direction finding system capable of estimating the bearing of an emitter using angle of arrival (AoA) phase interferometry. The current proposed system uses four receiver channels, each providing an analog signal that must be digitized and processed while preserving the relative phase information between channels. For data acquisition and processing, we selected the AMD Kria KR260, primarily because it was already available to us and offered extensive processing capabilities and I/O options. However, much of this I/O will prove to be challenging to use.

Our initial design used an AD9228 analog to digital converter operating at 50 MSPS per channel with a dedicated Spartan-7 FPGA responsible for acquiring and processing the samples before transmitting them to the KR260. Our revised design instead uses an LTC2324-16 operating at 2 MSPS per channel, removing the need for a secondary FPGA and allowing the KR260 to receive the ADC output directly. This has its drawbacks, the substantial reduction in sample rate meant the RF front end had to down convert the incoming RF signal more aggressively. However, the reduction in sample rate also created several advantages.

Notably, it simplified the custom PCB complexity and also enabled the use of cheaper traces reducing overall cost and implementation risk. The reduction in sample outputs also substantially reduces the outgoing data to the KR260, eliminating the need and complexity of multi gigabit communication and an additional FPGA.

Despite the substantial reduction in sampling rate, the revised ADC produces four simultaneous 16-bit data streams, for a total raw throughput of 128 Mb/s. The ADC has two selectable digital output interfaces, CMOS SPI compatible serial interface where each ADC channel has its own serial data output, and LVDS SPI compatible serial interface, where each output uses low voltage differential pairs.[3] Both interfaces pose those their own issues, including ADC clocking, serial output timing, and overall compatibility with the KR260. These issues are the driving motivation behind a deeper examination into the data acquisition architecture before the receiver hardware is finalized.

II. Technical Background

A. ADC Data Acquisition and Serial Interfaces

Each incoming analog IF signal is converted into a 16-bit digital sample. The LTC2324-16 samples all four channels simultaneously at up to 2 MSPS per channel. The samples must then be transferred to

the KR260 through one of the provided interfaces onboard the ADC. The serial interface used by the LTC2324-16 is determined by the input signal on pin 25, GND enables the CMOS interface while driving it to OV_{DD} enables the LVDS interface. Both of these interfaces are SPI compatible and feature a single data rate mode(SDR) and double data rate mode (DDR). The data rate mode runs SDR when driven to GND and DDR when driven to OV_{DD}. [3]

1) LVDS Data Output: LVDS mode uses Low Voltage differential signaling using two wires per signal. The receiver would detect the voltage difference between the two conductors. Differential signaling provides improved resistance to common mode interference by transmitting data over a pair of signal lines. This allows noise present in both lines to be rejected on the receiver end as the final output is measured by their potential difference. This is why LVDS is a popular interface for high speed ADC's [5].

However, LVDS poses a few implementation problems. LVDS may increase PCB complexity and cost, as differential traces must be carefully routed to maintain signal integrity and the PCB must be at a minimum 4 layers. Each differential pair should maintain a controlled impedance, typically around 100Ω, with matching termination at the receiver to minimize signal reflections. LVDS traces also have stricter routing criteria, including consistent spacing, closely matched trace length within pairs, and minimizing sharp bends. [6]

The LVDS connection also needs a 100Ω resistors across each differential pair at the receiver. However, the KR260 introduces a larger problem as the carrier board has no LVDS SPI connector. For 4 output lanes, an LVDS interface requires a total of 6 differential pairs. 4 pairs will serve as the differential data lines, 1 pair for the differential output clock, and one pair for the differential serial clock input. [3]

Despite not having a dedicated LVDS connector, the KR260 is equipped with ample GPIOs, notably it is equipped with 4 PMOD connectors, and a raspberry pi header. However, both the raspberry pi header and PMOD connectors use the High Density (HD) FPGA I/O Banks that only supports LVDS_25 inputs. Meaning the KR260 is unable to drive the pair for the differential serial clock input. The inputs also do not have internal 100Ω resistors so an LVDS implementation would require an external clock driver, differential routing constraints, and termination requirements [1][11].

While theoretically you can use an LVDS receiver to convert incoming differential signals into CMOS signals and an LVDS driver to convert outgoing CMOS signals into differential LVDS outputs. This adds both risk, and complexity, notably around timing. [1].

Any conversion integrated circuit (IC) would introduce a delay between its input and output. Even if LVDS pairs are routed directly into PMOD pins, we face other constraints. The PMOD connectors expose a limited number of complete differential pairs. Since the ADC requires at least five, at least two PMOD headers would be necessary. Routing the SDO signals and CLKOUT through seperate connectors introduces additional length differences and potential clock to data skew, increasing the complexity of meeting the FPGA's setup and hold timing requirements. This is compounded with the LVDS interfaces tighter data rate margins when compared to the CMOS interface. [3] [11]

So while LVDS would normally be an ideal form of serial communication the limitations of both the carrier board and PCB complexity makes it a poor fit for this project.

---

2) CMOS Data Output: CMOS mode uses a single ended signaling, with each data lane being a single wire. The ADC supports one, two, or four CMOS output lanes. Using more lanes reduces the serial clock frequency needed for a given conversion frequency. The four lane configuration use SDO1-SDO4, which is pin 27, 29, 35, and 39 respectively. The output voltage is configurable through OV_{DD}, which can be set to either 1.8V or 2.5V. For this project 2.5V will be selected as OV_{DD} to better meet electrical constraints. [3]

While the major challenge with LVDS was hardware compatibility the onboard I/O banks have more extensive support for CMOS signals. Meaning the KR260 can both directly receive the ADCs incoming CMOS signals and also drive the serial data clock inputs.

However, there are still many challenges facing a CMOS implementation including electrical compatibility, serial data acquisition, and timing reliability. The carrier board only exposes two types of GPIOs, a raspberry pi header and 4 Digilent PMOD 2x6 expansion headers. Both will have near identical electrical properties [9].

The first major concern that needs to be addressed is the electrical compatibility of the two devices as both the raspberry pi and PMOD headers implement the LVCMOS33 IO standard that uses $3.3V$ as logical high. The issue emerges from the disconnect with the LTC2324-16 that operates on $2.5V$ logic. However, the guaranteed output high ADC CMOS voltage is $OV_{DD} - 0.2V$ which for the selected voltage would be $2.3V$. The guaranteed output low ADC CMOS voltage is $0.2V$. AMDs publishes the DC characteristics for the ports and list the minimum voltage recognized as logical high at $2.0V$. This means theoretically the ADC high output should still be recognized as logical high by the KR260. However, the FPGA needs to generate the CLK input serial clock on the ADC and since both the PMOD and pi headers are tied to V_{CCO} and implementing the LVCMOS33 standard, both headers output at 3.3V, and this can not be changed. [10][3]

This means that a CMOS implementation will require a level translator, as $3.3V$ exceeds the LTC2324-16 maximum input voltage. These devices are relatively simple making it easy to add to the PCB. The current proposed IC is SN74AXC2T45 a two channel direction controlled logic level translator. This IC does introduce a small propagation delay on the serial clock input of the ADC between 0.5ns and 4ns. However, since the ADC generates its serial data output with a phase matched CLKOUT in response to the received serial clock, this delay doesn't reduce the available set up and hold timing margins. [12].

This leads into the discussion regarding the timing reliability risk of the CMOS implementation. Despite the ADC serial outputs being SPI compatible it is not the conventional four wire SPI. In CMOS mode 7 total wires are used CNV, SCK, CLKOUT, and 1 SDO for each channel. The SDO and CLKOUT are roughly phase aligned, and are intentionally out of phase with the SCK provided by the FPGA. Standard SPI interfaces have 4 signals, SCLK, MOSI, MISO, and CS/SS. SCLK is serial clock controlling when data bits are transmitted and received, and the ADC equivalent is the SCK. MOSI or master out, slave in, is the wire that send data from the master controller to peripheral which the ADC interface does not implement. Master in, Slave out is the wire that sends data from the peripheral to the controller, which is the SDO equivalent on the ADC interface. While the ADC interface doesn't contain a CS, or chip select pin, it does contain a CNV pin however its purpose differs from CS in SPI which selects which peripheral SPI talks to. As the CNV is a pulse that controls when the ADC begins capture and conversion. Unlike conventional SPI, the ADC interface also outputs a CLKOUT that is used to help the FPGA know when to capture data. [3][7]:

The LTC2324-16 has two data rate modes that can be selected SDR and DDR. The ADC data outlines key characteristics in addition to values I computed

Characteristics	CMOS SDR	CMOS DDR
Clock Frequency	110 Mhz	55 Mhz
Data transferred	1 bit per cycle	2 bits per cycle
Clock Cycles per 16 bit sample	16	8
interval rates	9.09 ns	9.09 ns
Maximum ADC throughput	2 MSPS/channel	2 MSPS/channel

Both approaches have an identical raw throughput of a 128 Mb/s, the primary difference stems from when data is transmitted. In SDR mode a bit is only sent out on the rising edge, while in DDR mode a bit is sent out on both the rising and falling edge which is why they have identical interval rates despite DDR having half the serial clock frequency. While this does reduce the clock frequency the FPGA has to generate, it also introduces possible setup and hold violations. [3]

The benefit of SDR mode is that data transitions occur on the falling edge of the serial clock then samples on the rising edge. This provides a sampling edge of 4.55 ns after data transition, providing a

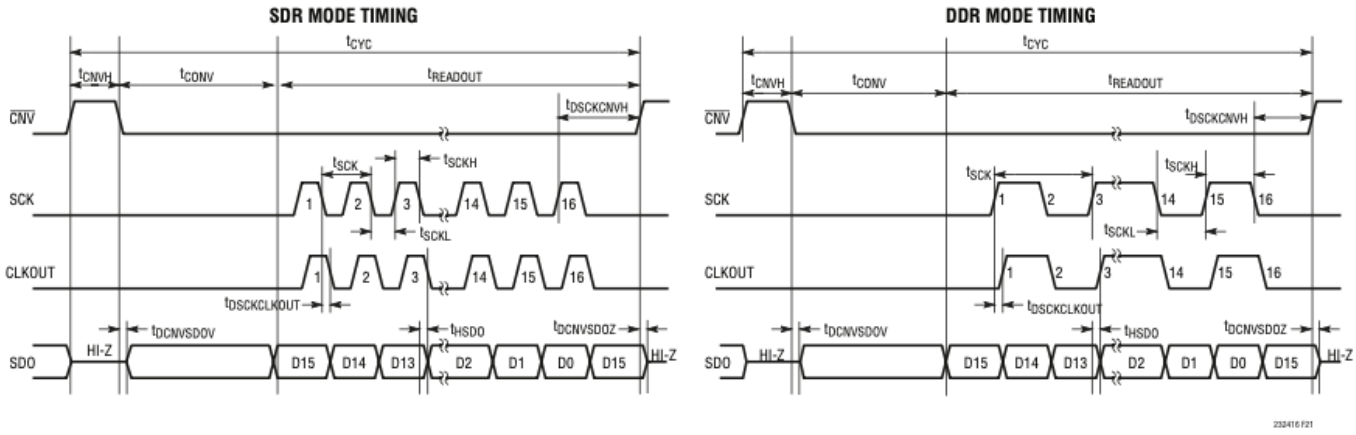


Fig. 1. CMOS SDR DDR timing diagram.

good margin for skew, and delays without violating the set up and hold delay. However, when operating in DDR mode the FPGA attempts to sample the output at the same time as a data transition. Since SDO and CLKOUT are digitally phase aligned DDR mode requires the SDO signal to be digitally delayed in order to provide the FPGA adequate setup and hold timing margins. However, this is where the limitations of the KR260 is reached, as the HD I/O Bank does not support a digital programmable delay [3][1].

Therefore, CMOS SDR is the preferred implementation for this design. Although SDR requires a higher serial clock frequency of 110 Mhz compared to DDR's 55 Mhz, its more favorable clock to data alignment eliminates the need for programmable delay recommended by Analog Devices for DDR operation, reducing implementation complexity while still achieving the ADC's full throughput.

III. FPGA Serial Data Capture and Synchronization

Beyond confirming the electrical compatibility and clock timing, the FPGA must correctly capture and reconstruct the ADC serial output into parallel data. Wan et al. investigates this problem using a low cost FPGA platform for a 12 bit 50 MSPS ADC operating with a 600 MSPS serial output. To address this, the researchers developed correction logic that stores previously received data in registers and then combines bits from consecutive periods to restore the original word [5].

Another synchronization approach is by Ngo and Nguyen who investigate FPGA based ADC acquisition when dedicated clock capable inputs are unavailable. Their implementation internally reconstructs the clock and frame signals using mixed mode clock management (MMCM) and input serializer and deserializer (ISERDES) resources. ISERDES enables real time sequential phase observation while the MMCM can adjust the internal clock phase dynamically [4].

Although both approaches address synchronization problems, their correction mechanisms may not be necessary. Unlike the system used by Ngo and Nguyen, the KR260 provides clock capable inputs that can potentially receive the ADCs returned CLKOUT directly. Furthermore, the LTC2324-16 provides a clock source synchronized with its serial data output, allowing the FPGA to capture on the rising edge of CLKOUT in SDR mode. If a consistent data stream can be captured with consistent timings Wan et al.'s approach may be unnecessary. [3].

The proposed implementation will use four 16-bit shift registers to reconstruct the ADC's simultaneous conversion results with a bit counter identifying the completion of each serial word. Unlike Wan et al.'s correction algorithm, these register would perform conventional serial to parallel conversion rather than compensate for misaligned words. This however makes the implementation susceptible to missed bit errors. However, this simpler architecture avoids the additional synchronization resources required by both methods.

Characteristics	CMOS SDR	CMOS DDR
Objective	Correct misaligned ADC words	Sychronize ADC clock, frame, and data
Problem	Incorrect word boundaries caused by timing differences	Clock phase misalignment without a clock capable inputs
solution	Register buffering and bit rearrangement	MMCM phase adjustment, ISERDES with bitslip primitives
FPGA Resources	SelectIO deserializer, registers, and correction logic	MMCM, ISERDES, phase control logic
Advantage	Recovers misaligned words	Dynamically align clock sample and data
Limitations for KR260	Correction may not be needed with a consistent ADC output	Complexity is unnecessary with a clock capable input
Use Case	Supports serial to parallel reconstruction	Provides an alternative if clock capture fails

While neither approach is expected to be necessary for the initial implementation, both provide potential solutions for specific data capture issues. The proposed design will instead use the LTC2324-16's synchronized CLKOUT to capture data in CMOS SDR mode, with four 16 bit shift registers reconstructing the parallel samples. The reliability of this approach will need to be verified experimentally through FPGA timing analysis and experimentation with phase alignment and word correction techniques from the two papers considered if they prove to be necessary.

IV. AI DISCOLSURE

OpenAI ChatGPT-6 was used to brainstorm technical approaches. This report was written using LaTeX overleaf, AI was used to fix missing package dependencies and formatting issues such as the tables not word wrapping.

REFERENCES

[1] Advanced Micro Devices, Inc., *UltraScale Architecture SelectIO Resources User Guide*, UG571, Jan. 2025. [Online]. Available: https://docs.amd.com/v/u/en-US/ug571-ultrascale-selectio [Accessed: Nov. 4, 2026].

[2] Advanced Micro Devices, Inc., *Zynq UltraScale+ Device Technical Reference Manual*, UG1085, Mar. 2025. [Online]. Available: https://docs.amd.com/v/u/en-US/ug1085-zynq-ultrascale-trm [Accessed: Nov. 4, 2026].

[3] Analog Devices, Inc., *LTC2324-16: Quad, 16-Bit, 2Msps/Ch Simultaneous Sampling ADC*, datasheet, Jun. 2017. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/232416f.pdf [Accessed: Nov. 4, 2026].

[4] V. M. Ngo and T. D. Nguyen, "A Reproducible FPGA–ADC Synchronization Architecture for High-Speed Data Acquisition," *Data*, vol. 11, no. 1, Art. no. 23, 2026, doi: 10.3390/data11010023. [Accessed: Nov. 5, 2026].

[5] Z. Wan, X. Hou, W. Jin, and S. Wu, "A Low-Cost Test Method for the Serial Output of ADC Based on FPGA," in *Proc. 2025 8th Int. Conf. Electronics Technology (ICET)*, Chengdu, China, 2025, pp. 746–752, doi: 10.1109/ICET64964.2025.11102934. [Accessed: Nov. 5, 2026].

[6] Texas Instruments, *LVDS SerDes Gen I PCB and Interconnect Design-In Guidelines*, Application Report SNLA008A, May 2004, rev. Aug. 2018. [Online]. Available: https://www.ti.com/lit/an/snla008a/snla008a.pdf [Accessed: Nov. 7, 2026].

[7] M. Usach Merino, "SPI Interface," Analog Devices, Inc., Application Note AN-1248. [Online]. Available: https://www.analog.com/en/resources/app-notes/an-1248.html [Accessed: Nov. 7, 2026]..

[8] A. Weiler, A. Pakosta, and A. Verma, "High-Speed Layout Guidelines," Texas Instruments, Application Report SCAA082A, Nov. 2006, rev. Aug. 2017. [Online]. Available: https://www.ti.com/lit/an/scaa082a/scaa082a.pdf [Accessed: Nov. 7, 2026]..

[9] Advanced Micro Devices, Inc., *Kria KR260 Robotics Starter Kit User Guide*, UG1092, rev. 1.1, Apr. 2024. [Online]. Available: https://docs.amd.com/r/en-US/ug1092-kr260-starter-kit [Accessed: Nov. 7, 2026].

[10] Advanced Micro Devices, Inc., *Zynq UltraScale+ MPSoC Data Sheet: DC and AC Switching Characteristics*, DS925, ver. 1.30, Jul. 2026. [Online]. Available: https://docs.amd.com/r/en-US/ds925-zynq-ultrascale-plus [Accessed: Nov. 7, 2026].

[11] Xilinx, Inc., "Kria KR260 Robotics Starter Kit Carrier Board Definition," *XilinxBoardStore*, board file ver. 2.2, 2022. [Online]. Available: https://github.com/Xilinx/XilinxBoardStore/blob/40fad97cf36f3d856c6759d83c2390de1566ffa7/boards/Xilinx/kr260_carrier/1.0/board.xml. [Accessed: Nov. 7, 2026].

[12] Texas Instruments, "SN74AXC2T45 2-Bit Translating Transceiver with Configurable Level-Shifting," datasheet SCES880D, rev. D, Jan. 2024. [Online]. Available: https://www.ti.com/lit/ds/symlink/sn74axc2t45.pdf. [Accessed: Nov. 7, 2026].